DAT (Dossier d'architecture technique)



Dossier d'architecture technique
I. Introduction
A. Contexte technique
1. Produit et périmètre
B. Objectifs du document
II. Architecture du système
A. Vue d'ensemble
1. Architecture proposée
2. Séparation des responsabilités
B. Composants du système
C. Diagramme d'architecture
1. Vue de déploiement
2. Flux principal : connexion Google
3. Séquence de connexion étudiant
4. Logique de décision à la connexion
III. Spécifications techniques
A. Langages de programmation
1. Critères de choix : performance, maintenance et tests
2. Comparaison des solutions frontend
3. Comparaison des solutions backend
B. Services tiers
C. Base de données
1. Modèle conceptuel (MCD)
2. Modèle logique (MLD)
3. Modèle physique (MPD)
D. Sécurité
1. Protection des donnée personnelles (RGPD)
2. Connexion exclusive par Google : conséquences
E. Compatibilité et accessibilité
IV. Plan de tests
A. Stratégie de test
1. Intégration à Scrum
2. Niveaux de tests
B. Outils de test
C. Cahier de recettes
1. Compte de test
2. Scénarios de recette
V. Plan de maintenance
A. Stratégie de maintenance
1. Types de maintenance
2. Entretien régulier
B. Documentation
VI. Sources
1. Architecture et choix techniques

2. Sécurité, RGPD et connexion Google
3. Accessibilité
4. Test et maintenance

Dossier d’architecture technique

I. Introduction

A. Contexte technique

1. Produit et périmètre

Projet Globe est la refonte de l'intranet d'un organisme de formation. Il comporte un espace étudiant et un espace pédagogique. Il couvre l'authentification, les projets, les inscriptions, les rendus, les profils, les promotions, les unités de formation, les groupes, les compétences, les temps de log et les statistiques. Les accès dépendent du rôle et du périmètre pédagogique de chaque utilisateur.

Les références de travail sont dans le “Cahier des charges”.

B. Objectifs du document

Ce dossier fournit une base commune à l'équipe de développement, au directeur technique, au Product Owner, aux testeurs et aux personnes chargées de l'exploitation. Il doit permettre de :

- comprendre les responsabilités des composants et leurs échanges ;
- justifier les technologies proposées et leurs limites ;
- relier chaque fonction à un module, des données et une stratégie de tests ;
- définir les contrôles d'accès, d'intégrité, de disponibilité et de confidentialité ;
- préparer les diagrammes de classes et de base de données demandés ;
- organiser les déploiements, la maintenance et l'onboarding après un an ;
- identifier les arbitrages nécessaires avant une mise en production.

Décision d’architecture	Proposition	Justification
ADR-01	Monolithe modulaire	Transactions cohérentes et exploitation initiale simple
ADR-02	Angular et TypeScript pour le client	Conventions communes, formulaires typés et outillage homogène pour un intranet riche
ADR-03	Go et `net/http` pour l'API	Services explicites, interfaces ciblées, outils de tests et serveur HTTP standard
ADR-04	PostgreSQL	Relations, intégrité et agrégations pédagogiques
ADR-05	pgx, SQL paramétré et migrations SQL versionnées avec Goose proposé	Contrôle explicite des relations, transactions et calculs pédagogiques
ADR-06	Session applicative opaque côté serveur	Révocation et droits courants contrôlables
ADR-07	REST JSON et OpenAPI	Contrats explicites et intégrations lisibles
ADR-08	Client et API sous une même origine publique	Simplifie cookies et politique d'origine
ADR-09	Rendus par liens initialement	Réduit le stockage et le traitement de fichiers
ADR-10	Conteneurs et PostgreSQL administré si budget adapté	Reproductibilité et réduction de la charge d'exploitation

II. Architecture du système

A. Vue d'ensemble

1. Architecture proposée

Nous proposons un **monolithe modulaire à trois couches principales** : présentation web, application serveur et persistance. Le client Angular affiche les deux espaces ; l'API Go applique les règles métier ; PostgreSQL stocke les données et garantit les relations. Un reverse proxy sert les ressources statiques, termine HTTPS et transmet les requêtes `/api` au serveur.

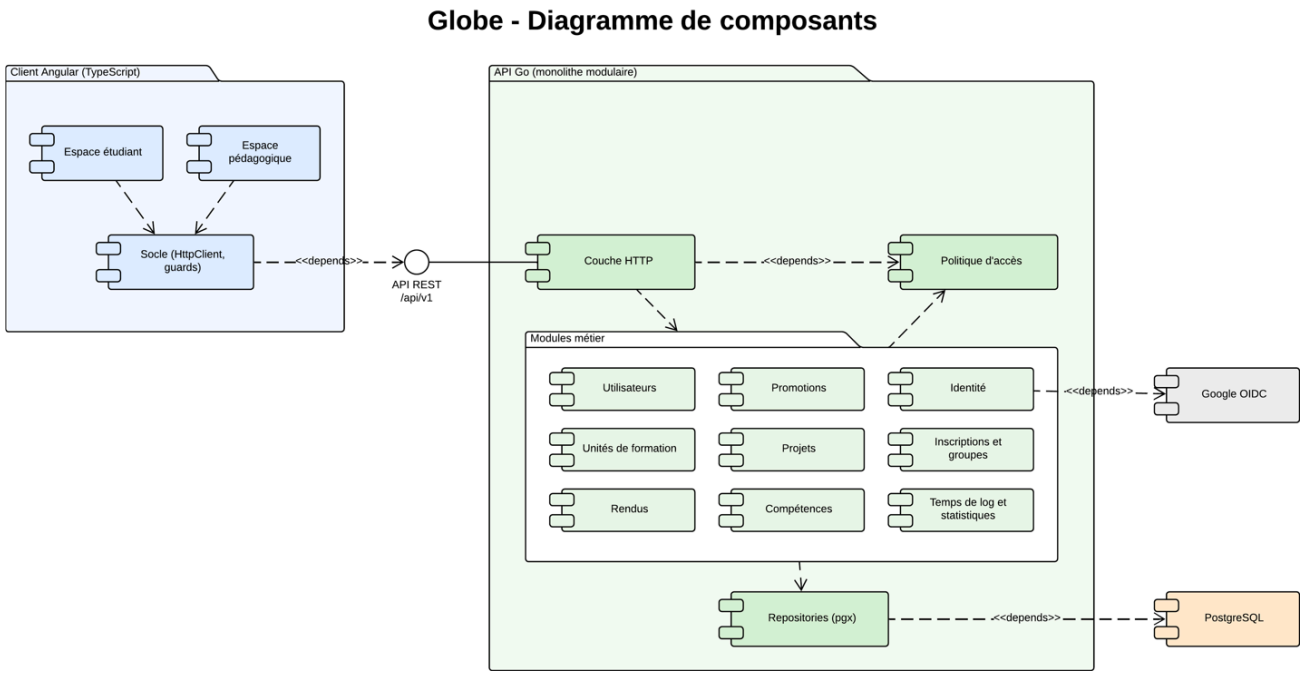
Le navigateur ne dialogue jamais directement avec la base et ne contient aucun secret Google ou secret d'infrastructure. Les contrôles visuels facilitent l'usage mais ne constituent pas une protection. L'API vérifie l'identité, les habilitations et les conditions métier de chaque opération.

Ce choix répond à des domaines liés : inscription, groupe, rendu et suivi utilisent des données communes. Les microservices ajouteraient dès le départ des communications réseau, des contrats entre services, des transactions distribuées et davantage d'exploitation, sans volume ni besoin d'autonomie établis. Une séparation en services pourra être étudiée si les mesures ou l'organisation la justifient. Le monolithe n'interdit pas plusieurs instances de l'API.

2. Séparation des responsabilités

- Les composants affichent des DTO et gèrent les interactions, sans recalculer les droits de référence.
- Les handlers décodent et valident le contrat HTTP, puis délèguent aux services ; ils ne portent pas directement les requêtes SQL et règles pédagogiques.
- Les politiques évaluent l'action, le rôle, l'objet, le périmètre et l'état courant du compte.
- Les services portent les règles, les transactions et les changements d'état.
- Les repositories encapsulent les requêtes SQL paramétrées et le mapping vers les structures Go. Les interfaces sont définies aux endroits où elles isolent réellement les dépendances ; pas d'interface ou de repository générique obligatoire par table.
- Les adaptateurs isolent Google, les sources de logs et de compétences, et le stockage éventuel.

B. Composants du système



Le diagramme de composants présente l'organisation logicielle de Globe en deux grands ensembles. Le premier est le client Angular, qui regroupe l'espace étudiant et l'espace pédagogique. Tous deux s'appuient sur un composant Socle qui centralise les appels HTTP, les guards de routes et l'état de session.

Le second est l'API Go, conçue comme un monolithe modulaire. Les deux ensembles ne communiquent que par une interface unique, l'API REST /api/v1, fournie par la couche HTTP de l'API. Le client ne connaît donc ni la base de données ni les règles métier : il consomme uniquement un contrat documenté avec OpenAPI.

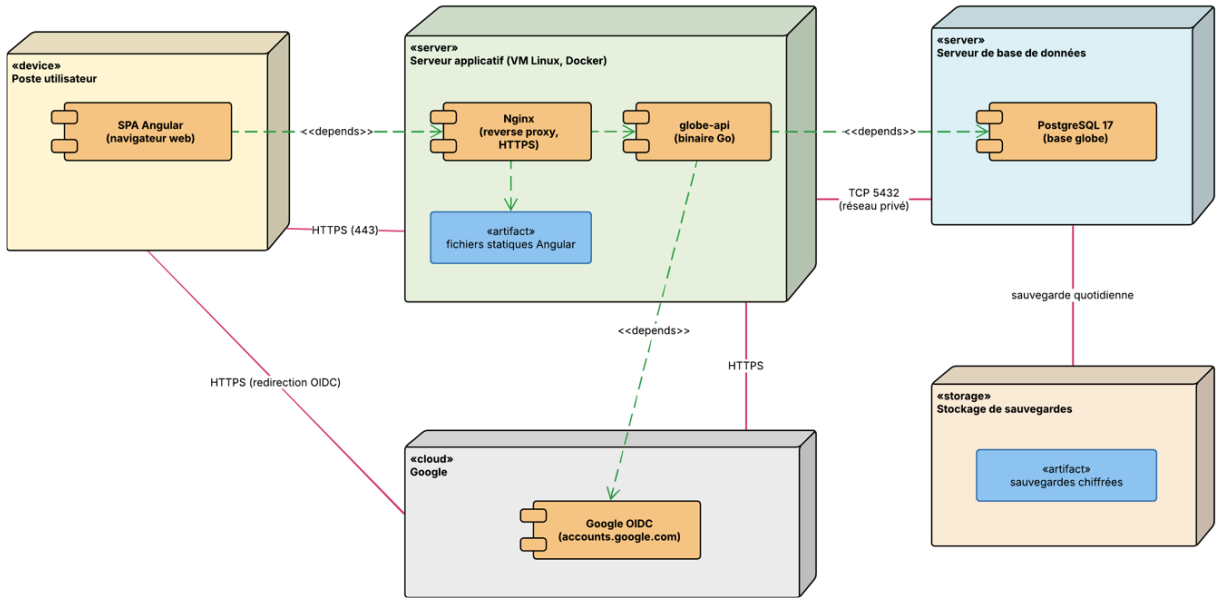
Dans l'API, chaque requête suit le même chemin. La couche HTTP valide la session, le jeton CSRF et les données reçues. Elle transmet ensuite la demande au module métier concerné : identité, utilisateurs, promotions, unités de formation, projets, inscriptions et groupes, rendus, compétences, ou temps de log et statistiques.

Avant toute opération, les modules interrogent la politique d'accès, seul endroit où sont évalués le rôle et la promotion de l'utilisateur. Ils accèdent aux données uniquement via les repositories, qui exécutent des requêtes SQL paramétrées sur PostgreSQL avec pgx.

Seul le module Identité dépend d'un service externe, Google OIDC, pour l'authentification. Grâce à ce découpage, chaque module peut être testé isolément et un nouveau domaine peut être ajouté sans modifier les autres. Les règles d'autorisation restent elles aussi concentrées en un seul point.

C. Diagramme d'architecture

Globe - Diagramme de déploiement



1. Vue de déploiement

Le diagramme de déploiement montre l'installation physique de Globe sur quatre nœuds :

- le poste utilisateur, où la SPA Angular s'exécute dans le navigateur ;
- le serveur applicatif, une VM Linux avec Docker qui héberge Nginx, les fichiers statiques Angular et le binaire globe-api ;
- le serveur de base de données PostgreSQL ;
- le stockage de sauvegardes.

Google intervient comme service externe pour l'authentification.

Seul le serveur applicatif est exposé à Internet, en HTTPS sur le port 443. Nginx termine le chiffrement, sert l'application Angular et transmet les requêtes /api à l'API Go.

La base de données reste sur un réseau privé : seule l'API y accède, en TCP sur le port 5432.

Le navigateur ne communique donc jamais directement avec la base et ne détient aucun secret. Une sauvegarde chiffrée est réalisée chaque jour vers un stockage séparé, et sa restauration est testée régulièrement.

2. Flux principal : connexion Google

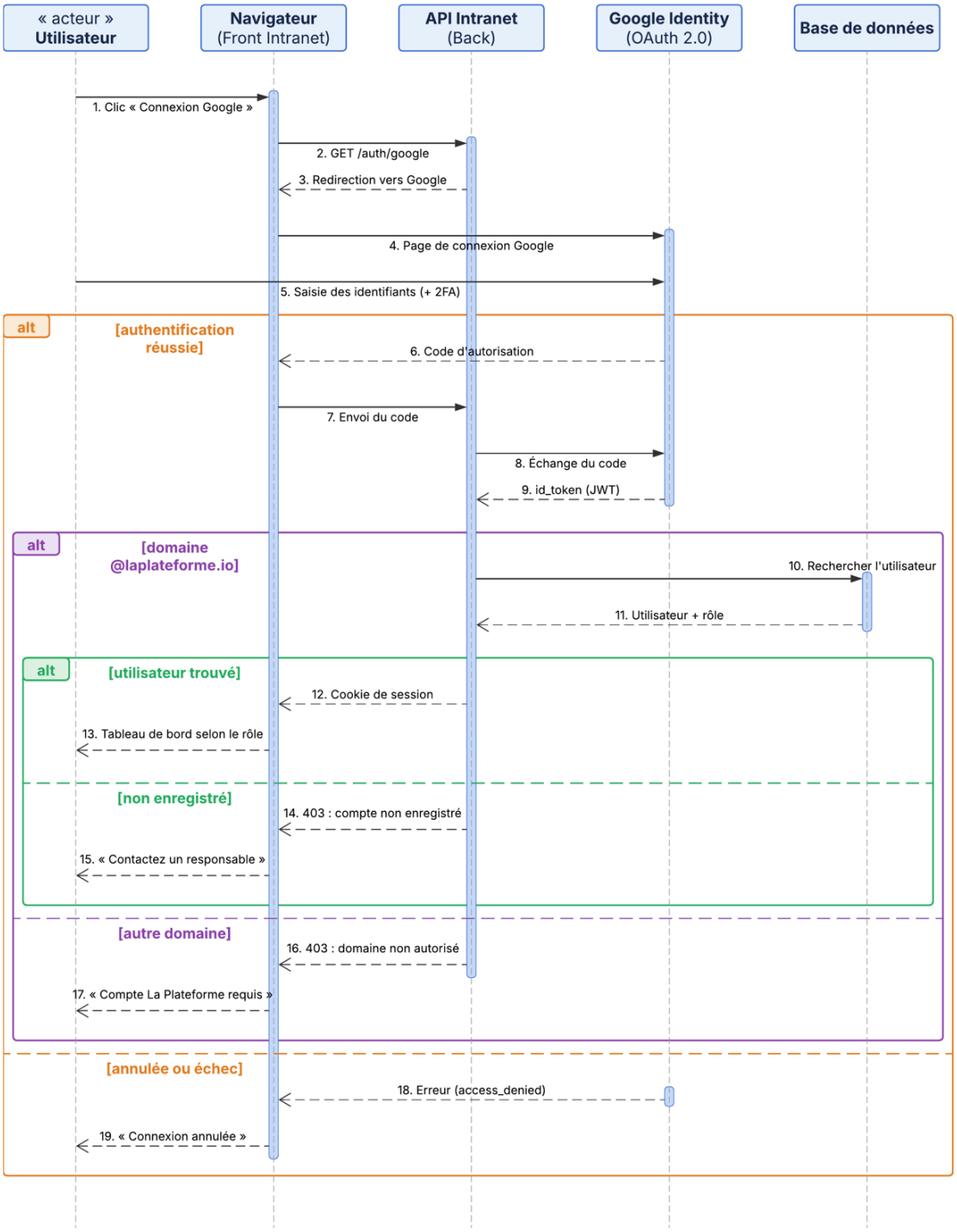
Lors de la connexion, le navigateur est redirigé vers Google, puis revient sur l'API avec un code d'autorisation.

L'API échange ce code directement avec Google, vérifie l'identité obtenue, contrôle que le compte est autorisé, puis crée une session côté serveur.

Le navigateur ne reçoit qu'un cookie de session sécurisé : aucun jeton Google n'est stocké côté client.

3. Séquence de connexion étudiant

Diagramme de séquence : connexion à l'intranet Globe via Google SSO



Ce diagramme met en évidence un choix central : l'**authentification est déléguée à Google**, mais l'**autorisation reste entièrement dans Globe.**

Google confirme seulement l'identité de la personne. C'est l'API qui décide si cette personne a le droit d'entrer et avec quel rôle. Ainsi, Globe ne stocke aucun mot de passe et profite de la double authentification de Google.

Un compte Google valide ne suffit pas pour accéder à l'intranet : il faut aussi être enregistré par l'équipe pédagogique.

L'échange du code contre le jeton se fait d**e serveur à serveur**, entre l'API et Google. Le navigateur ne voit jamais le **id_token** et ne reçoit qu'un cookie de session **HttpOnly**, inaccessible au JavaScript. Une **faille XSS** dans le front ne permettrait donc pas de voler l'identité de l'utilisateur.

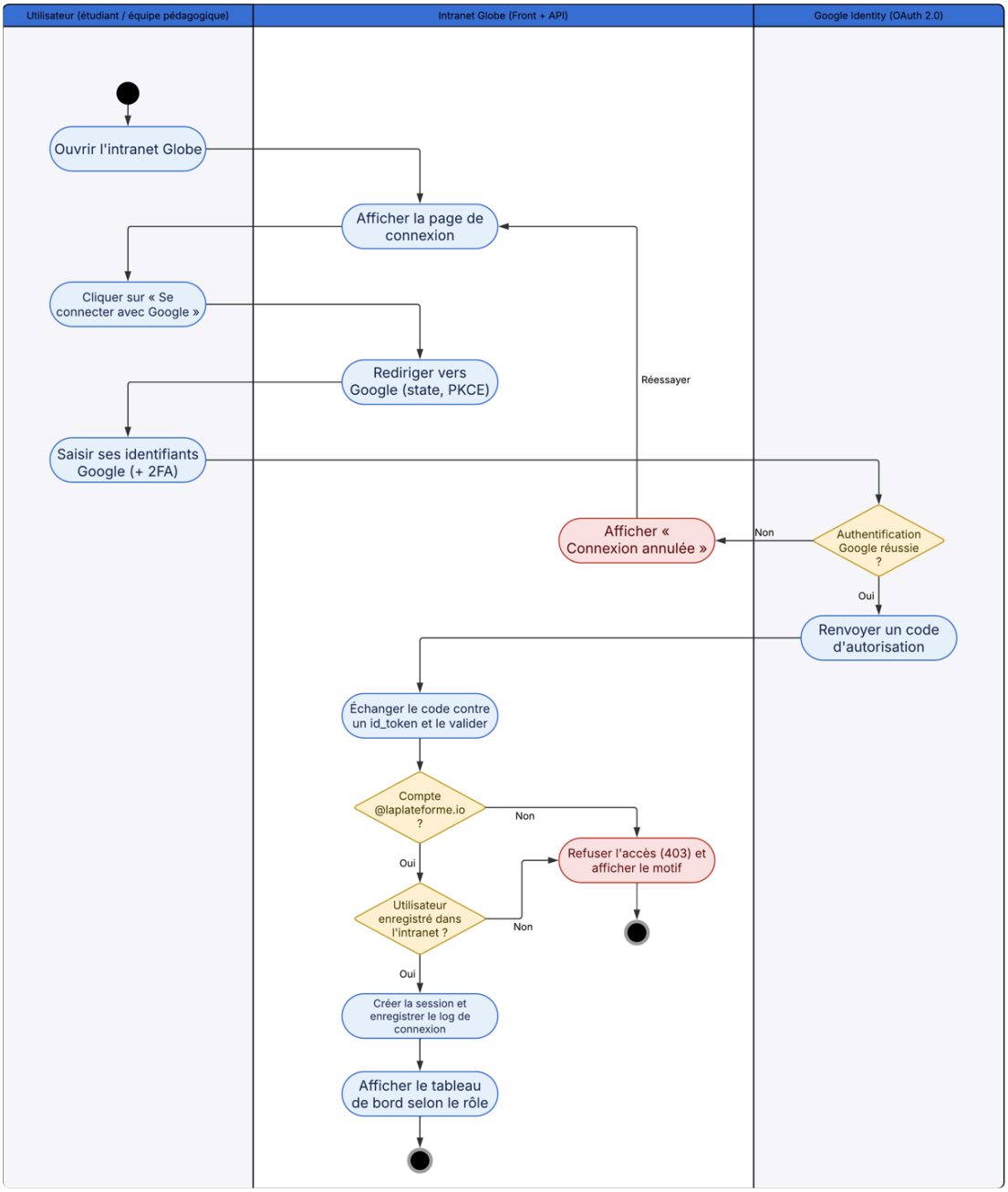
Comme la session est stockée côté serveur, un compte désactivé ou un rôle retiré prend effet dès la requête suivante, sans attendre l'expiration d'un jeton.

Les fragments alt montrent enfin que les trois cas d'échec (domaine refusé, compte inconnu, connexion annulée) se terminent tous sans création de session.

Ces cas constituent directement nos scénarios de tests d'intégration sur l'authentification. Les messages d'erreur restent volontairement génériques afin de ne pas révéler si un compte existe dans l'intranet.

4. Logique de décision à la connexion

Diagramme d'activité : connexion à l'intranet Globe via Google SSO



Le diagramme d'activité fait apparaître **l'ordre des contrôles**, qui n'est pas arbitraire. Le domaine **@laplateforme.io** est vérifié avant toute recherche en base : une personne extérieure est rejetée sans que Globe ne consulte ni n'expose ses données.

La session n'est créée qu'en dernière étape, une fois toutes les conditions validées. Il n'existe donc aucun état intermédiaire où un utilisateur serait connecté sans rôle confirmé.

Les couloirs montrent aussi la **répartition des responsabilités**.

- **L'utilisateur** agit uniquement sur Google pour s'authentifier.
- Toutes les **décisions d'accès** se trouvent dans le couloir de Globe, **côté serveur.**
- Le **front** n'a aucun pouvoir de décision : masquer un bouton ou une page relève du confort, pas de la sécurité.

Du point de vue de l'expérience utilisateur, chaque échec mène à un **message explicite et à une issue** : réessayer après une annulation, ou contacter un responsable lorsque le compte n'est pas enregistré.

Cela limite les demandes au support. Le diagramme montre aussi un point à arbitrer avec le métier : l'inscription des comptes reste manuelle. Une synchronisation automatique avec la liste des étudiants de La Plateforme pourrait être envisagée dans une version ultérieure.

III. Spécifications techniques

A. Langages de programmation

1. Critères de choix : performance, maintenance et tests

Globe comporte des formulaires nombreux, des règles d'accès, des historiques et des statistiques ; il doit évoluer longtemps et accueillir de nouveaux développeurs. Nous évaluons donc les solutions selon :

- la lisibilité des règles métier et les garanties de typage
- la cohérence des conventions et la facilité de reprise du code
- l'isolation des responsabilités et les possibilités de tests unitaires et d'intégration
- les performances observables sous une charge réaliste
- le coût des requêtes, du rendu, de la mémoire et de l'exploitation
- le support, les mises à jour, les bibliothèques et les compétences disponibles

2. Comparaison des solutions frontend

Solution	Atouts pour Globe	Limites et coûts	Position dans la proposition
Angular / TypeScript	Ensemble intégré de routage, formulaires, injection et outils ; conventions favorables aux écrans métier et au travail collectif	Apprentissage plus exigeant, discipline RxJS/signals et migrations régulières ; bundle et rendu à surveiller	**Retenu** pour structurer un intranet riche sur la durée
React / TypeScript	Écosystème large et composition flexible ; tests possibles avec un outillage adapté	Davantage de décisions à harmoniser autour des formulaires, état, routage et conventions selon la stack choisie	Alternative crédible si l'équipe dispose d'une architecture React maîtrisée
Vue / TypeScript	Composants et réactivité adaptés au même besoin ; adoption souvent progressive	Gouvernance à définir pour formulaires, état et structure ; la souplesse n'est ni une faute ni une garantie de maintenance	Alternative crédible, non retenue par défaut sans comparaison

Angular est préféré ici pour réduire les choix d'assemblage et homogénéiser les nombreux écrans de gestion, pas pour une supériorité absolue de vitesse.

3. Comparaison des solutions backend

Solution	Atouts pour Globe	Limites et coûts	Position dans la proposition
ASP.NET Core / C#	Typage, injection, pipeline HTTP, politiques d'autorisation et outillage intégré ; accès asynchrones et tests d'intégration	Compétences .NET nécessaires ; ORM et requêtes à surveiller ; deux langages avec le frontend	Alternative crédible, non retenue : nous privilégions ici des packages Go et une composition explicite
Spring Boot / Java	Configuration, composants et écosystème adaptés aux applications métier ; tests et exploitation outillés	Expertise Spring/JPA nécessaire ; configuration, démarrage et consommation à mesurer selon déploiement	Alternative de premier plan si l'équipe est plus compétente en Java
NestJS / TypeScript	Langage commun avec le client ; modules et injection ; adapté à une API d'intranet	Attention aux traitements CPU et tâches longues dans le processus Node ; validation runtime indispensable	Alternative sérieuse si l'expertise TypeScript réduit fortement le coût de maintenance
Go et bibliothèques HTTP	Langage compilé, outils de test et modèle de concurrence ; services explicites	Davantage d'assemblage pour les conventions métier, permissions et données selon les bibliothèques choisies	**Retenu** pour un backend explicite, testable et déployé comme un binaire ; performances à mesurer

Django / Python	ORM, administration et mécanismes web intégrés ; bonne base pour des workflows CRUD	Architecture API à préciser ; typage et besoins CPU à examiner ; administration générique non équivalente à l'espace pédagogique	À étudier si rapidité de réalisation et compétences Python dominent

Ces solutions ne sont pas éliminées pour une incapacité supposée à tester ou à évoluer. Les différences portent sur les conventions, le coût d'assemblage, les compétences et le profil de charge. Nous ne prétendons pas que Go surpasse chaque alternative sur Globe sans mesure.

B. Services tiers

Service	Usage dans Globe	Justification
Google OIDC	Authentification des utilisateurs (comptes	Comptes déjà fournis par l'école, double authentification, aucun mot de pass stocké
Stockage object S3 (Scaleway, OVH ou AWS S3)	Documents, images, péces jointes des projets et rendus	Stockage illimité, peu coùteaux, accès par URL signée temporaire
Service d'e-mails	Notifications : inscription validée, rendu reçu, échéance proche	Délivrabilité gérée, pas de serveur mail à maintenir
Sentry	Remontée des erreurs du front Angular et de l'API Go	Erreurs regroupées avec leur contexte, alertes en temps réel
Grafana Cloud (Prometheus, Loki)	Metriques, logs et tableaux de bord d'exploitation	Surveillance de la disponibilité et des performances
ClamAV	Analyse antivirus des fichiers déposés	Empêche la diffusion de fichiers malveillants entre utilisateurs
Github (Actions, Container Registry)	Code source, CI/CD, images Docker	Tests et déploiements automatisés à chaque modification

Globe s'appuie sur quelques services externes pour les fonctions qui ne relèvent pas de son cœur métier. L'objectif est de réduire le code à maintenir et de bénéficier d'outils éprouvés.

Ces services sont tous isolés derrière l'API Go : le navigateur ne communique jamais directement avec eux, à l'exception de la redirection vers Google lors de la connexion.

Les clés d'accès restent côté serveur. Les fournisseurs hébergés dans l'Union européenne sont privilégiés pour les données personnelles, conformément au RGPD.

Chaque service est encapsulé dans un adaptateur dédié, ce qui permet de le remplacer par une alternative sans impacter les modules métier.

C. Base de données

Pour le projet Globe on va utiliser PostgreSQL comme base unique. Les données de l'intranet sont fortement liées entre elles : une inscription relie un étudiant à un projet, un rendu relie une inscription ou un groupe à un projet, et une statistique croise promotions, logs et compétences. Une base relationnelle garantit ces liens par des contraintes, au lieu de les confier uniquement au code.

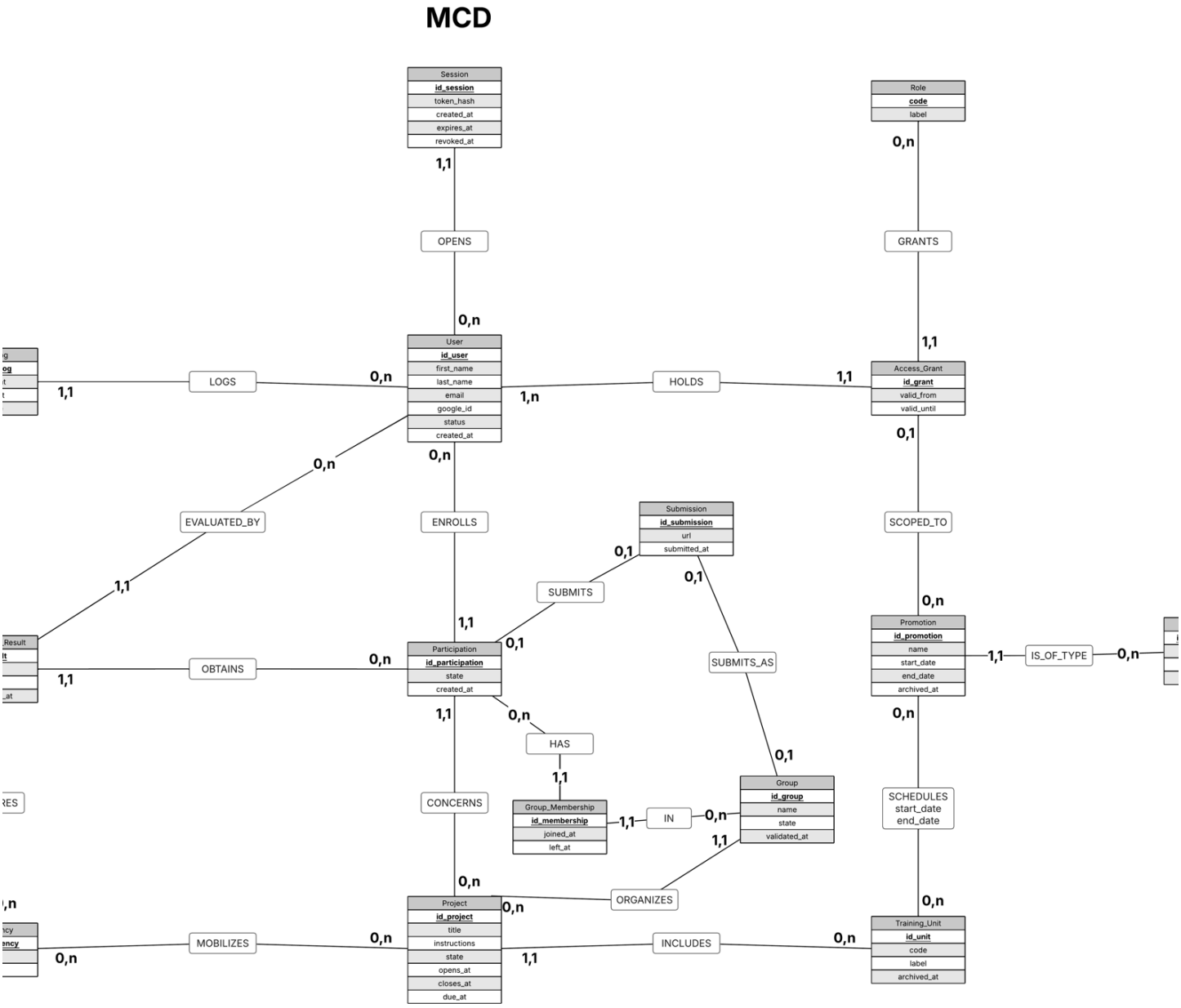
1. Modèle conceptuel (MCD)

Le MCD repose sur un choix structurant : il n'existe qu'un seul type d'utilisateur.
Étudiants, responsables, accompagnateurs et administrateurs sont tous des USER. Leur rôle est porté par une habilitation (ACCESS_GRANT) associée, si nécessaire, à une promotion. Tout le monde se connecte donc par le même parcours Google, et un changement de rôle ou de promotion ne nécessite pas de créer un nouveau compte. Comme chaque habilitation a une date de début et de fin, l'historique est conservé : on sait dans quelle promotion se trouvait un étudiant à une date donnée.

L'organisation pédagogique distingue la promotion et l'unité de formation. Une même unité peut être suivie par plusieurs promotions, et une promotion suit plusieurs unités : l'association SCHEDULES est donc de type n,n et porte les dates de début et de fin de l'unité pour cette promotion.

Chaque projet appartient en revanche à une seule unité de formation. Les types de promotion (bachelor, MSc, alternance…) sont modélisés par une entité PROMOTION_TYPE, qui décrit le libellé, le mode de formation et la durée. Chaque promotion est d'un seul type. Ajouter un nouveau type revient donc à insérer une ligne, sans modifier le schéma.

Un étudiant s'inscrit à un projet par une PARTICIPATION. C'est à partir d'elle que se rattachent son groupe, son rendu et ses résultats de compétences. Chaque résultat est lié à l'inscription évaluée et à l'utilisateur qui l'a saisi (EVALUATED_BY), ce qui permet de savoir dans quel projet une compétence a été évaluée, et par qui. L'appartenance à un groupe (GROUP_MEMBERSHIP) est une entité à part entière, datée par une entrée et une sortie, afin de conserver l'historique des changements de groupe.



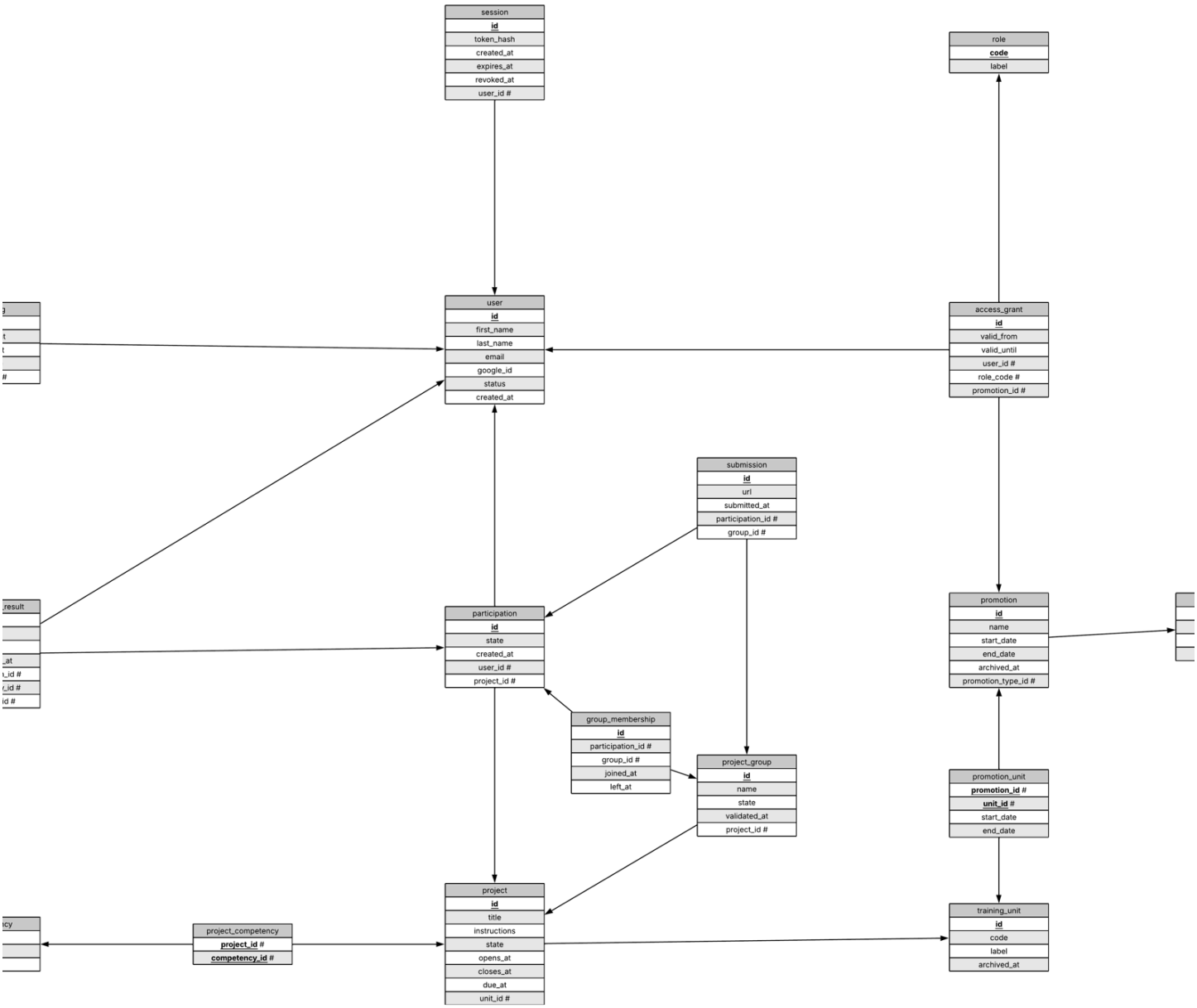
2. Modèle logique (MLD)

Les associations 1,n deviennent des clés étrangères, par exemple project.unit_id ou promotion.promotion_type_id. Les associations n,n deviennent des tables de jointure : promotion_unit, qui conserve les dates de l'association SCHEDULES, et project_competency. group_membership possède sa propre clé, car une même inscription peut changer de groupe au cours du projet.

Le MLD fait aussi apparaître les règles d'intégrité garanties par la base elle-même, indépendamment du code :

- unicité de l'inscription d'un étudiant à un projet, UNIQUE (user_id, project_id)
- un seul résultat par compétence et par inscription, UNIQUE (participation_id, competency_id)
- un seul groupe courant par inscription, par un index unique partiel sur left_at IS NULL
- appartenance du groupe et de l'inscription au même projet, vérifiée par un trigger
- exclusivité du propriétaire d'un rendu : une inscription ou un groupe, jamais les deux
- cohérence des dates (fin postérieure au début)
- bornes des résultats de compétences (0 à 100)

MLD



3. Modèle physique (MPD)

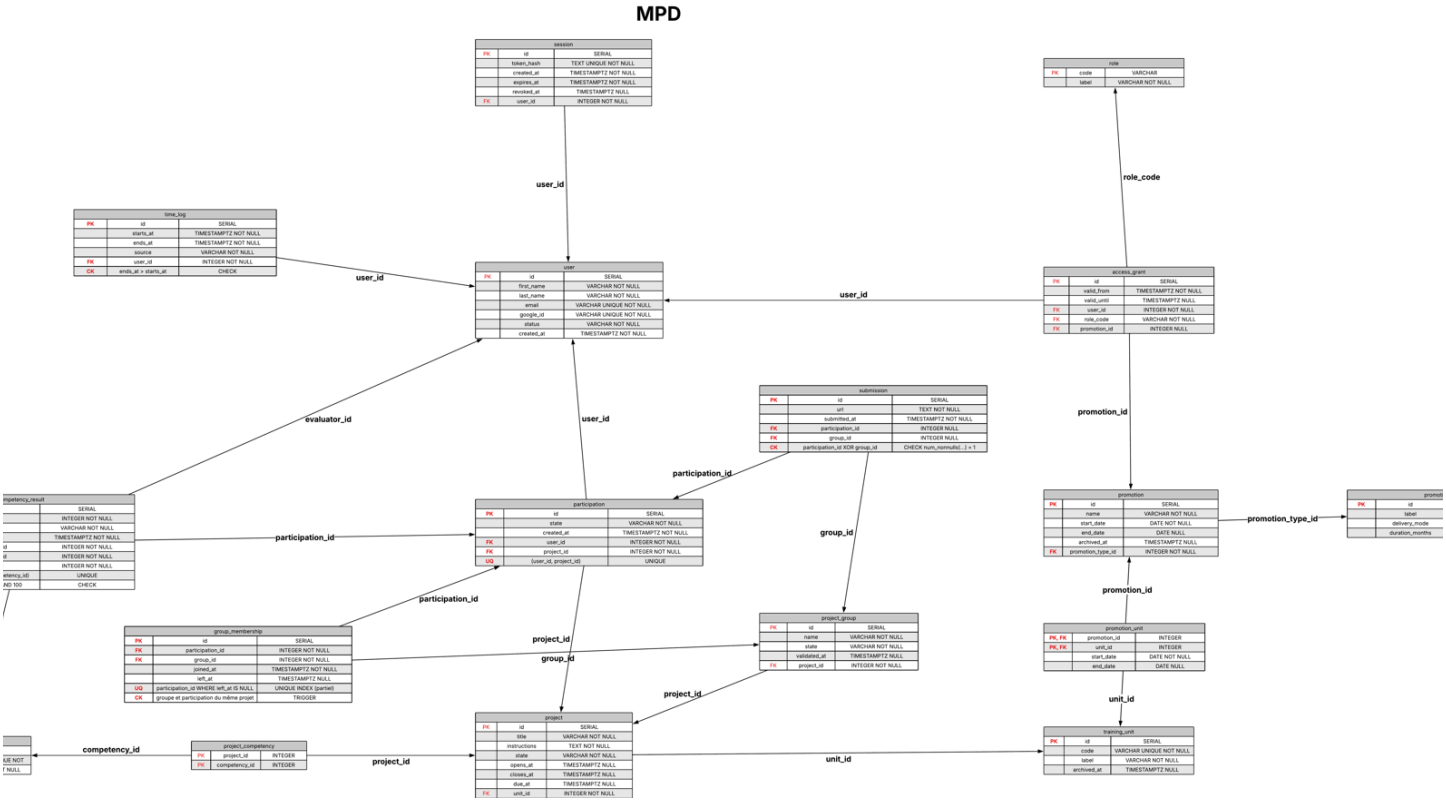
Le MPD traduit le modèle dans les types PostgreSQL :

- Instants : TIMESTAMPTZ, stockés en UTC, pour éviter les erreurs de fuseau horaire et de changement d'heure.
- Dates de promotion et de planification des unités : DATE, car ce sont des dates civiles et non des instants.
- Temps de log : uniquement le début et la fin. La durée se calcule (ends_at - starts_at) et n'est jamais stockée, ce qui évite qu'elle devienne incohérente avec les deux dates.

Côté sécurité, la table session ne contient qu'une empreinte du jeton et jamais le jeton lui-même : une fuite de la base ne permettrait pas d'usurper une session. L'identifiant Google (google_id) est unique et sert de référence à la connexion, plutôt que l'adresse e-mail, qui peut changer.

Enfin, toutes les clés étrangères utilisent ON DELETE RESTRICT : la suppression d'une promotion, d'une unité ou d'un étudiant ayant un historique est refusée au ieu d'effacer ses données en cascade.

Les promotions et les unités de formation qui ne sont plus utilisées sont archivées grâce à une date d'archivage (archived_at), ce qui préserve l'historique pédagogique. Le schéma évolue uniquement par migrations versionnées (Goose). Elles sont relues comme du code, testées en CI et appliquées de façon identique sur chaque environnement.



D. Sécurité

1. Protection des donnée personnelles (RGPD)

Globe traite les données personnelles des étudiants et du personnel pédagogique. Nous appliquons le principe de minimisation : chaque donnée a une finalité précise, et une donnée sans finalité n'est pas collectée.

Le tableau ci-dessous est notre proposition. Les durées doivent être validées par le référent RGPD de l'organisme.

Catégorie	Données (table)	Finalité	Qui y accède	Conservation proposée
Identité	Nom, prénom, e-mail, identifiant Google sub, statut (user)	Identifier la personne et la rattacher à sa promotion	L'étudiant, les encadrants de sa promotion, le superadmin	Durée de la formation + 1 an, puis supression ou anonymisation
Habilitations	Rôle, promotion, période de validité (access_grant)	Appliquer les droits d'accès	Le superadmin, les responsables pédagogiques	Comme l'identitè, pour guarder l'historique des parcours
Sessions	Empreinte du jeton, dates de création, d'expiration et de révocation (session)	Maintenir la connexion et pouvoir la révoquer	Personne (usage technique uniquement)	Suppression 30 jours après expiration
Parcours pédagogique	Inscriptions, groupes, rendus (liens HTTPS)	Suivre les projets	L'étudiant, son groupe, les encadrants de sa promotion	Durée de la formation + 1 an, sauf obligation liée à la certification
Compétences	Résultats, état brouillon ou validé (competency_result)	Évaluer la progression	L'étudiant, les encadrants, l'intervenant en mission	Durée de la formation + 1 an, sauf obligation liée à la certification
Temps de log	Début, fin, durée, source (time_log)	Suivre la présence et l'engagement	L'étudiant, son responsable, son accompagnateur	Durée de la formation + 1 an
Statistiques	Agrégats sans identifiant	Piloter l'activité (Direction)	La Direction	Aucune donnée individuelle stockée
Traces techniques	Acteur, action, objet, date	Sécurité et audit des actions sensibles	L'exploitation, sur un périmètre restreint	6 mois pour les logs techniques, 1 an pour l'audit

Sauvegardes	Copie chiffrée de la base	Reprise après incident	L'exploitation	30 jours glissants

Temps de log

Il s'agit de la donnée la plus sensible, car elle peut être perçue comme une surveillance. Globe enregistre uniquement des intervalles de temps (début, fin) et leur source. Il n'enregistre ni adresse IP, ni frappe clavier, ni capture d'écran, ni géolocalisation.
La Direction ne voit que des données agrègées, jamais le temps d'un etudiant donne. L'accompagnateur consulte les logs de sa promotion sans pouvoir les modifier. L'etudiant voit toujours ses propres logs.

Base légale et information

Le traitement repose sur l'exécution du contrat de formation. Une mention d'information est affichée sur la page de connexion, et une page « Données personnelles » est accessible depuis toutes les pages.

Ces deux contenus indiquent les finalités, les durées, les destinataires et le contact du référent RGPD.

Droits des étudiants

- L'étudiant consulte à tout moment ses données dans son profil (**droit d'accès**).
- Il demande la correction d'une donnée erronée (**droit de rectification**). Les champs administratifs restent modifiables uniquement par le personnel habilité.
- Il peut exercer ses droits d'effacement, de limitation et d'opposition. Ces droits sont limités tant que les données sont nécessaires au suivi de la formation ou à la certification.
- Les demandes sont adressées au référent RGPD, qui répond sous un mois (article 12 du RGPD).
- Les données réelles ne sont jamais utilisées en recette : les tests reposent sur des jeux de données fictifs.

Mesures de sécurité

- Les accès sont contrôlés par l'API sur chaque requête, avec un refus par défaut.
- Les jetons de session sont stockés sous forme d'empreinte.
- Les échanges passent en HTTPS et les sauvegardes sont chiffrées.
- Les comptes SQL sont distincts selon l'usage (migration, exécution, sauvegarde).
- Les journaux ne contiennent jamais ni jeton ni contenu de profil.

2. Connexion exclusive par Google : conséquences

Sécurité

- **Aucun mot de passe local.** Globe ne stocke aucun mot de passe : une fuite de la base ne contient donc aucun identifiant réutilisable.
- **Authentification déléguée**. La robustesse de l'authentification dépend de la politique Google Workspace de l'école (double authentification, verrouillage des comptes).
- **Admission vérifiée par Globe.** Une connexion Google réussie ne suffit pas. L'API vérifie elle-même la signature du jeton, l'émetteur, l'audience, le nonce et le domaine @laplateforme.io, puis l'existence d'une habilitation active.
- **Identifiant de rattachement**. Le compte est rattaché par l'identifiant Google sub, et non par l'adresse e-mail, qui peut changer.

Données

- **Ce que Globe demande à Google**. Globe demande uniquement les scopes openid email profile. Il reçoit l'identifiant, l'e-mail et le nom, et rien d'autre.
- **Ce que Globe conserve de Google**. Aucun jeton Google n'est conservé après la connexion, et aucun accès « hors ligne » n'est demandé. Globe crée ensuite sa propre session, révocable.
- **Rôle de Google**. Google intervient comme sous-traitant de l'école dans le cadre de Google Workspace for Education. L'encadrement des éventuels transferts hors UE relève du contrat entre l'école et Google.

Limites acceptées

- **Indisponibilité de Google.** Si Google est indisponible, personne ne peut se connecter, car il n'existe volontairement aucun accès de secours. Ce risque est accepté et communiqué aux utilisateurs.
- **Départ d'un étudiant**. Désactiver son compte Google bloque les nouvelles connexions, mais pas une session déjà ouverte. C'est pourquoi l'API vérifie le statut du compte et ses habilitations à chaque requête, et révoque les sessions lors d'une désactivation dans Globe.
- **Intervenants externes**. Ils n'ont pas de compte La Plateforme. Leur mode de connexion reste à décider

E. Compatibilité et accessibilité

Application responsive sur ordinateur et téléphone.
Matrice proposée : deux dernières versions stables de Chrome, Firefox, Edge et Safari au moment de chaque release, plus Safari iOS et Chrome Android sur

appareils de référence. Cette matrice doit être validée selon les usages de l'organisme ; les résultats sont datés et ne valent pas pour tous les appareils.

Les liens directs vers une page protégée passent par la connexion puis reviennent sur un chemin interne autorisé. Le proxy renvoie l'application pour les routes SPA, mais une route `/api` inconnue reste une erreur API, jamais un document HTML de remplacement. Les changements de version du client sont traités sans perdre un formulaire silencieusement.

Nous proposons WCAG 2.2 niveau AA comme cible de conception et de recette. Cela ne détermine pas à lui seul le référentiel légal applicable ni une conformité acquise.

Domaine	Dans Globe
Navigation	Parcours clavier, ordre cohérent, focus visible et non masqué
Formulaires	Labels, consignes et erreurs associées ; état non porté seulement par une couleur
Changements de vue	Titre, focus et annonces pertinentes
Statistiques	Tableau ou texte équivalent au graphique, unités et population
Affichage	Contraste, zoom, retour à la ligne et adaptation aux petites dimensions
Gestes	Alternative à glisser-déposer ou action exclusivement tactile

IV. Plan de tests

A. Stratégie de test

1. Intégration à Scrum

Les tests accompagnent les incréments dès le début.

- À **l'affinage**, le Product Owner et les Developers clarifient les critères d'acceptation et les décisions bloquantes
- En **Sprint Planning**, les Developers planifient aussi fixtures, contrôles et documentation.
- Pendant le **sprint**, les tests sont exécutés automatiquement sur les changements et manuellement sur les parcours concernés.
- La **Review** inspecte un résultat utilisable ; elle ne remplace pas la recette technique.
- La **rétrospective** traite les défauts de processus.

Definition of Done proposée :

- critères d'acceptation couverts,
- code relu,
- tests pertinents réussis,
- contrôles de types
- construction réussis
- absence de faille bloquante connue
- migrations testées
- documentation et recette mises à jour,
- incrément déployable.

2. Niveaux de tests

Niveau	Objet
Unitaires	Règles pures et services isolés
Composants client	Formulaires, états et interactions
Intégration	Modules et PostgreSQL réel
Contrat API	DTO, statuts et schémas OpenAPI
Bout en bout	Parcours dans un navigateur
Sécurité	Identité, session et autorisation

Performance	Charge représentative
Accessibilité	Automatique et manuel
Exploitation	Déploiement, sauvegarde et restauration

B. Outils de test

Nous avons choisi les outils de test les plus proches de notre stack, pour éviter d'ajouter des dépendances inutiles.

Côté serveur:

- le **package testing de Go**
- **httptest**

suffisent pour tester les règles métier, les permissions et les routes de l'API sans lancer de vrai serveur.
Les r**equêtes SQL** et les contraintes de la base

- sont testées sur une vraie base PostgreSQL, lancée dans un conteneur, parce qu'une base simulée ne vérifierait pas les règles d'intégrité.

Côté client

- es composants Angular sont testés avec les outils fournis par **Angular CLI.**

Les parcours complets (connexion, inscription à un projet, dépôt d'un rendu)

- sont automatisés avec **Playwright**, et axe-core détecte une partie des problèmes d'accessibilité.

Tous ces tests sont lancés automatiquement par la CI à chaque modification. Une modification n'est fusionnée que si tous ses tests passent. Les tests manuels sont décrits dans le cahier de recettes.

C. Cahier de recettes

Ce cahier de recettes décrit les scénarios qui permettent de vérifier que Globe répond aux exigences fonctionnelles **EF-01 à EF-15** du cahier des charges

Chaque scénario précise ses préconditions, les étapes à suivre et le résultat attendu, ce qui permet à n'importe quel membre de l'équipe ou au Product Owner de l'exécuter de la même façon.

1. Compte de test

Compte	**Role**	Périmètre	**Particularité**
`etu1@laplateforme.io`	Étudiant	Promotion P1	Compte principal des scénarios étudiants
`etu2@laplateforme.io`	Étudiant	Promotion P1	Second membre pour les groupes
`etu3@laplateforme.io`	Étudiant	Promotion P2	Sert aux tests de cloisonnement
`resp1@laplateforme.io`	Responsable	Promotion P1	
`acc1@laplateforme.io`	Accompagnateur	Promotion P1	
`dir1@laplateforme.io`	Direction	Promotion P1	
`admin1@laplateforme.io`	Superadmin	Global	
`inactive@laplateforme.io`	Étudiant	Promotion P1	Compte au statut `disabled`

2. Scénarios de recette

Authentification et session (EF-01, EF-15)

ID	Scénario	Etapes	Resultat attendu	Resultat obtenu	Status
REC-AUTH-01	Connexion d'un étudiant enregistré	1. Ouvrir Globe 2. Cliquer « Se connecter avec Google » 3. Choisir `etu1` et s'authentifier	Tableau de bord étudiant affiché avec le nom de `etu1`	-	Non exécuté
REC-AUTH-02	Refus d'un compte hors domaine	1. Lancer la connexion 2. Choisir `perso@gmail.com`	Message « Compte La Plateforme requis » ; aucune session créée	-	Non exécuté
REC-AUTH-03	Compte La Plateforme non enregistré	1. Lancer la connexion avec `nouveau`	Message « Contactez un responsable » ; aucun compte créé automatiquement	-	Non exécuté
REC-AUTH-04	Compte désactivé	1. Lancer la connexion avec `inactif`	Accès refusé ; le message ne révèle pas que le compte existe	-	Non exécuté
REC-AUTH-05	Annulation chez Google	1. Lancer la connexion 2. Annuler sur la page Google	Message « Connexion annulée » ; bouton pour réessayer ; aucune session	-	Non exécuté
REC-AUTH-06	Déconnexion	1. Cliquer « Se déconnecter » 2. Utiliser le bouton Retour du navigateur	Page de connexion affichée ; aucune donnée privée visible après le retour	-	Non exécuté
REC-AUTH-07	Session expirée	1. Tenter de s'inscrire à PR1 (projet 1)	Redirection vers la connexion ; l'inscription n'est pas enregistrée automatiquement	-	Non exécuté
REC-AUTH-08	Lien direct sans session	1. Ouvrir l'URL de PR1 (projet1)	Passage par la connexion puis retour sur PR1 (projet1)	-	Non exécuté

V. Plan de maintenance

A. Stratégie de maintenance

Globe doit rester utilisable et sûr pendant plusieurs années, avec une équipe qui change. La maintenance n'est donc pas une phase qui commence après la livraison : elle est intégrée aux sprints, avec la même Definition of Done que les nouvelles fonctionnalités.

1. Types de maintenance

Type	Exemples dans Globe	Comment elle est traitée
Corrective	Inscription impossible, statistique fausse, accès accordé à tort	Ticket Jira « Bug », correction, test de non-régression, livraison
Préventive	Mise à jour des dépendances, renouvellement des certificats, test de restauration, purge des sessions	Tâches planifiées à cadence fixe, voir le point 3
Adaptative	Changement côté Google, nouvelle version de navigateur, nouvelle règle de formation	Analyse d'impact, puis story dans le backlog
Évolutive	Nouveau type de promotion, nouvelle statistique	Story priorisée par le Product Owner, avec ses critères d'acceptation
Sécurité	Faille dans une dépendance, compte compromis, fuite de données	Traitée immédiatement, hors sprint si nécessaire

Responsabilités

- Le **Product Owner** arbitre les priorités.

- Les **Developers** analysent et corrigent.
- Le r**esponsable technique** veille à l'architecture, aux versions et à la dette technique.
- Le **référent RGPD** intervient dès qu'un incident touche des données personnelles.

2. Entretien régulier

Fréquence	Activités
En continu	Surveillance des erreurs (Sentry), des métriques et des alertes (Grafana)
Chaque sprint	Traitement des bugs et de la dette technique, mise à jour de la documentation
Chaque semaine	Revue des pull requests de mise à jour de dépendances ouvertes par Dependabot
Chaque mois	Mise à jour des images Docker, purge des sessions expirées, revue des coûts
Chaque trimestre	Revue des habilitations (comptes et rôles), test de restauration d'une sauvegarde
Avant chaque release	Vérification des migrations, de la compatibilité et du plan de retour arrière

B. Documentation

La documentation est versionnée et revue avec le code ou la régle modifiée.

Git conserve les sources techniques

Confluence peut publier les synthèses et décisions

Jira suit les travaux

Document	Contenu attendu	Mise à jour
README développeur	Prérequis, lancement, variables sans secrets et commandes	À chaque changement d'installation
Dossier d'architecture	Composants, flux, contraintes	À chaque changement structurant
OpenAPI	Routes, données, erreurs et authentification	Avec les contrats API
Modèle et dictionnaire de données	Cardinalités, champs, contraintes	Avec mligrations et décisions métier
Diagramme de classes	Services, entités, dépendances et interfaces	Avec évolution des modules
Cahier de recettes	Couverture et résultats réellement exécutés	À chaque incrément concerné
Guide d'onboarding	Organisation, code, données, tests et premier parcours	Vérifié lors de l'arrivées d'un développeur
Storybook		
Runbook	Déploiement, retour, restauration, révocation et incidents	Après changement
ADR (Architecture decision record)	Choix, alternatives, statut et conséquences	À chaque décision

Sentry, Grafana Cloud, le service d'e-mails et le stockage S3 reçoivent des données techniques ou personnelles.

Nou choisissons pour chacun une région hébergée dans l'Union européenne. Sentry est configuré pour masquer les données personnelles (e-mails, noms, cookies) avant l'envoi. Les e-mails de notification contiennent uniquement le nom de l'étudiant et le titre du projet, jamais un résultat ni une évaluation.

Après un an, l'onboarding présente l'état réel : versions livrées, décisions acrives, dette, incidents et évolutions evisagées. Le nouvel arrivant doit pouvoir lancer l'evvironnement, exécuter un test et suivre une foncionnalité du composant Angular au handler Go, puis au service et à la requete SQL.

VI. Sources

1. Architecture et choix techniques

Source	Lien
ADR, Architecture Decision Records	Architectural Decision Records (ADRs)
Angular, documentation	Home • Angular
Go, documentation	Documentation - The Go Programming Language
Go : package net/http	http package - net/http - Go Packages
pgx, driver PostgreSQL pour Go	GitHub - jackc/pgx: PostgreSQL driver and toolkit for Go
Goose, migrations	GitHub - pressly/goose: A database migration tool. Supports SQL migrations and Go functions.
Spécification OpenAPI	OpenAPI Specification v3.2.1
Nginx, documentation	nginx documentation

2. Sécurité, RGPD et connexion Google

Source	**Lien**
Google : OpenID Connect	OpenID Connect | Sign in with Google | Google for Developers
OpenID Connect Core 1.0	Final: OpenID Connect Core 1.0 incorporating errata set 2
RFC 7636 : PKCE	RFC 7636: Proof Key for Code Exchange by OAuth Public Clients
OWASP : Session Management Cheat Sheet	Session Management - OWASP Cheat Sheet Series
OWASP : Authorization Cheat Sheet	Authorization - OWASP Cheat Sheet Series
OWASP Top 10	OWASP Top 10 | OWASP Foundation
CNIL : Les durées de conservation des données	Les durées de conservation des données
Sentry : masquage des données personnelles	Data Scrubbing | Sentry Docs
AWS S3 : URL présignées	Sharing objects with presigned URLs - Amazon Simple Storage Service

3. Accessibilité

Source	**Lien**
RGAA	Référentiel général d'amélioration de l'accessibilité - RGAA
W3C : WCAG 2.2	Web Content Accessibility Guidelines (WCAG) 2.2
Angular : accessibilité	Accessibility • Angular

4. Test et maintenance

Source	Lien
Go : package testing	testing package - testing - Go Packages
Go : package httptest	httptest package - net/http/httptest - Go Packages
Testcontainers for Go	Testcontainers for Go
Playwright	Fast and reliable end-to-end testing for modern web apps | Playwright
axe-core	GitHub - dequelabs/axe-core: Accessibility engine for automated Web UI testing
GitHub Actions	GitHub Actions documentation - GitHub Docs
Dependabot	Securing your supply chain - GitHub Docs
Sentry, documentation	Sentry Docs | Application Performance Monitoring & Error Tracking Software
Grafana, documentation	Technical documentation | Grafana Labs
ClamAV, documentation	Introduction - ClamAV Documentation
PostgreSQL : sauvegarde et restauration	Chapter 25. Backup and Restore